

FUNCTIONS

① Basics of Functions

A function is a reusable block of code that performs a specific task when called. Functions are useful to organize code, make it reusable, and reduce redundancy.

② Defining a Function

You define a function using the `def` keyword followed by the function name, parentheses, and a colon.

Syntax

```
def function_name(parameters):  
    # Block of code
```

Example: Basic function to greet a user

```
def greet():
```

```
    print("Hello! welcome to the Python course!")
```

```
greet()
```

Output

Hello! welcome to the Python course.

③ Function Parameters

Parameters are variables used to pass data into a function.

Example: Function with a Parameter

```
def greet - user (name):
```

```
    print ("Hello, {name}! welcome to the Python course.
```

```
greet - user ("Ancud")
```

Out put

Hello, Ancud! welcome to the Python course.

④ Returning value from a Function

A function that adds two numbers, and returns the result

```
def add - numbers (a, b):
```

```
    return a+b
```

```
result = add - numbers (10, 20)
```

```
print ("The sum is:", result)
```

output

(The sum is: 30)

⑤ Default Parameter values

You can define a default value for a Parameter, which is used if no argument is passed when the function is called.

Example: Function with a Default Parameter

```
def greet (name = "student"):
```

```
    print ("Hello, {name}! welcome to the Python course.
```

greet () # uses default value "student"

greet ("Gecta") # uses passed value "Gecta"

Output

Hello, student! welcome to the Python course.

Hello, Geetha! welcome to the Python course.

Here are the Section for nested Functions and local/global variables:

⑥ Local and Global variables

- Local variables are defined inside a function and only accessible within that function.
- Global variables are defined outside all functions and are accessible from anywhere in the code.

Example: Local vs Global variables.

```
name = "Global name"
```

```
def greet():
```

```
    name = "Local name"
```

```
    print(name)
```

```
greet() # Print local variable
```

```
print(name) # Prints global variable.
```

Output

Local name

Global name

In this example, the local variable name inside the function, does not affect the global variable name.

1) Greet function write a function greet() that takes on no arguments and prints a greeting message.

Functions - A advanced concepts

In this part, we'll explore more advanced function concepts including recursion, lambda functions, and variable length arguments.

① Keyword Arguments

with keyword arguments. You can pass values to a function by specifying the parameter names.

Example

```
def display_info(name, age):  
    print(f"Name: {name}, Age: {age}")
```

```
display_info(age=25, name="Kumar")
```

Output

name: Kumar, Age: 25

② Variable - Length Arguments

You can use `*args` and `**kwargs` to accept a variable number of arguments in a function.

Example: Using `*args`

```
def total_sum(*numbers):  
    result = 0
```

for num in numbers:

~~result += num~~

for num in numbers:
result += num

return result

Print (total-sum (1, 2, 3, 4))

output

110

Example: using **kwargs

def student-info (**details):

for key, value in details.items():

Print (f' {key}: {value}')

Student info (name="Anand", age=22, course="Python")

output

name: Anand

age: 22

course: Python

(3) Lambda Functions

A lambda function is a small anonymous function that can take any number of arguments but has only one expression.

Syntax:

lambda arguments: expression

Example Lambda function to double a number

double = lambda x: x * 2

Print (double(5))

output

10

④ Recursion

Recursion occurs when a function calls itself. It's used to solve problems that can be broken down into smaller, similar problems.

Example: Recursive function to calculate factorial

```
def factorial(n):
```

```
    if n == 1:
```

```
        return 1
```

```
    else:
```

```
        return n * factorial(n-1)
```

```
print(factorial(5))
```

Output: 120

Here are the sections for nested functions and Local/Global variables:

⑤ Nested Functions

A nested function is a function defined inside another function. The inner function is only accessible within the outer function, allowing for more modular and controlled code execution.

Example: nested function in python

```
def outer_function(name):
```

```
    def inner_function():
```

```
        print("Hello, " + name + "!")
```

```
outer_function("Anand")
```

Output

Hello, Anand.

In the example, the inner_function() is called within outer_function() and name parameter of the outer function